

JavaScript Complete Handbook

PART 1: Fundamentals & Core Language

Absolute Beginner to Strong Intermediate · Interview Ready · 175+ Q&A

INTRO & HISTORY	VARIABLES	DATA TYPES	OPERATORS
TYPE COERCION	CONTROL FLOW	FUNCTIONS	SCOPE
HOISTING	ARRAYS	STRINGS	OBJECTS
175+ Q&A	CHEAT SHEETS	BEST PRACTICES	REAL EXAMPLES

13 Chapters · 100 Beginner Q&A · 75 Intermediate Q&A · Complete Cheat Sheets · 2026

TABLE OF CONTENTS

01	Introduction to JavaScript	History · ECMAScript · JS Engine · JIT Compilation
02	Variables & Data Types	var · let · const · 8 data types · Scope · Hoisting
03	Operators	Arithmetic · Comparison · Logical · Ternary · Nullish
04	Type Conversion	Implicit coercion · Explicit conversion · Famous traps
05	Control Flow	if/else · switch · for · while · do-while · break/continue
06	Functions	Declaration · Expression · Arrow · IIFE · Internals
07	Scope	Global · Function · Block · Scope chain · Closure intro
08	Hoisting	Variable hoisting · Function hoisting · TDZ
09	Arrays	map · filter · reduce · sort · slice · splice · flat · more
10	Strings	All important methods · Template literals · Interview Q&A
11	Objects	Creation · Properties · Methods · Destructuring · Spread
12	Interview Questions	100 Beginner + 75 Intermediate with detailed answers
13	Cheat Sheet & Revision	Complete quick-reference for all chapters

**CHAPTER
01**

Introduction to JavaScript

From a weekend scripting language to the world's most-used programming language — where JS came from and how it works.

1.1 What Is JavaScript?

JavaScript is a **high-level, interpreted, dynamically-typed, single-threaded, prototype-based** programming language. It is the only language natively supported by all major web browsers, making it the core language of the web. Today JavaScript runs everywhere: browsers, servers (Node.js), mobile apps (React Native), desktop apps (Electron), IoT devices, and even databases.

1.2 History of JavaScript

1995	Brendan Eich at Netscape creates 'Mocha' in 10 days. Renamed to LiveScript, then JavaScript for marketing.
1996	Microsoft creates JScript for IE. Competing implementations cause fragmentation.
1997	JavaScript submitted to ECMA International. First ECMAScript standard (ES1) published.
1999	ES3 — introduced regular expressions, try/catch, and string methods.
2009	ES5 — strict mode, JSON support, Array methods (forEach, map, filter, reduce).
2015	ES6 / ES2015 — the big leap: let/const, arrow functions, classes, Promises, modules, destructuring, template literals.
2016+	Yearly releases: ES2016–ES2024 add async/await, optional chaining, nullish coalescing, private fields, and more.
2009	Node.js by Ryan Dahl — JavaScript on the server using V8 engine. Revolution begins.

1.3 ECMAScript vs JavaScript

ECMAScript is the official specification maintained by ECMA International (TC39 committee). **JavaScript** is the implementation of that specification — plus browser/runtime-specific APIs (DOM, fetch, setTimeout). Think of ECMAScript as the recipe and JavaScript as the dish.

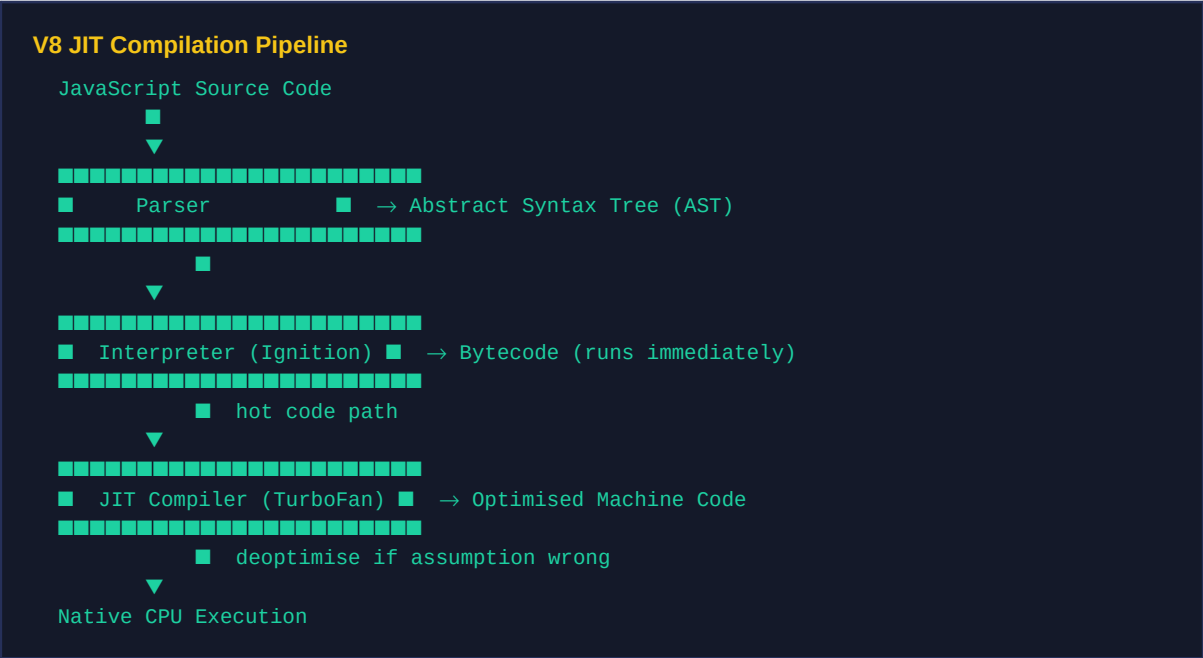
1.4 JavaScript Engine

A JavaScript engine reads and executes JavaScript code. Each browser has its own engine. They all implement the ECMAScript spec but may differ in optimisations.

Engine	Used By	Key Feature
V8	Chrome, Node.js, Edge	JIT, TurboFan optimising compiler
SpiderMonkey	Firefox	First JS engine, Mozilla
JavaScriptCore	Safari, React Native	Nitro optimiser
Hermes	React Native	Optimised for mobile startup

1.5 JIT Compilation

JavaScript is neither purely interpreted nor purely compiled. Modern engines use **Just-In-Time (JIT) Compilation**: the engine compiles hot code paths to native machine code at runtime, while interpreting cold code. This gives performance near compiled languages.



1.6 Browser vs Node.js

Feature	Browser	Node.js
Environment	Client-side	Server-side
Global object	window	global / globalThis
DOM access	Yes (document, window)	No DOM
File system	No (sandboxed)	Yes (fs module)
Modules	ES Modules (native)	CommonJS + ES Modules
Use case	UI, user interaction	APIs, CLI tools, servers
Runtime	Bundled with browser	Installed separately

1.7 Interview Questions

Q: What is the difference between JavaScript and ECMAScript?

ECMAScript is the specification/standard. JavaScript is the implementation. ECMA International's TC39 committee defines ECMAScript. JavaScript engines (V8, SpiderMonkey) implement it. JavaScript also includes runtime-specific APIs (like DOM in browsers, fs in Node.js) that are not part of ECMAScript.

Q: Why is JavaScript called a single-threaded language?

JavaScript runs on a single call stack — only one piece of code executes at any given time. There is no parallel execution of JS code (unlike Java with threads). Non-blocking behaviour is achieved through the event loop, which offloads I/O operations to the browser/OS while the main thread stays free.

Q: What is JIT compilation and how does it benefit JavaScript?

JIT (Just-In-Time) compilation converts hot JavaScript code to native machine code at runtime. The engine identifies frequently-executed code paths (hot spots) and compiles them for speed, while interpreting cold code immediately. This gives JavaScript performance close to compiled languages without sacrificing dynamic flexibility.

CHAPTER SUMMARY

- ✓ JavaScript was created in 1995 and standardised as ECMAScript
- ✓ JavaScript engines use JIT compilation for near-native performance
- ✓ V8 powers Chrome, Node.js, and Edge — most common JS engine
- ✓ Browser JS has DOM access; Node.js has file system, no DOM
- ✓ ECMAScript is the spec; JavaScript is the implementation + runtime APIs

PRACTICE QUESTIONS

1. Name three JavaScript engines and which environments they are used in.
2. What happens between writing JavaScript code and CPU execution?
3. List two things available in the browser but not in Node.js, and vice versa.
4. What does it mean that JavaScript is 'dynamically typed'?

**CHAPTER
02**

Variables & Data Types

var, let, const — and the 8 data types that form the foundation of every JavaScript program.

2.1 Variable Declaration: var, let, const

Feature	var	let	const
Scope	Function	Block	Block
Hoisted	Yes (undefined)	Yes (TDZ)	Yes (TDZ)
Redeclaration	Allowed	Not allowed	Not allowed
Reassignment	Allowed	Allowed	Not allowed
Global property	Yes (window.x)	No	No
When to use	Legacy code only	Mutable values	Constants/refs

```
// var – function scoped, avoid in modern code
var x = 10;
var x = 20; // redeclaration: ALLOWED (confusing!)
x = 30;     // reassignment: allowed

// let – block scoped, use for mutable values
let count = 0;
// let count = 1; // SyntaxError: already declared
count = 1;     // reassignment: allowed

// const – block scoped, use for everything else
const PI = 3.14159;
// PI = 3; // TypeError: assignment to constant

// const with objects: reference is constant, NOT the value
const user = { name: 'Alice' };
user.name = 'Bob'; // ALLOWED – mutating the object
// user = {}; // TypeError – can't reassign the reference
```

2.2 Scope Demonstration

```
var a = 'global';
let b = 'global-let';

function test() {
  var a = 'function'; // new 'a' in function scope
```

```

let b = 'block';    // new 'b' in block scope

if (true) {
  var a = 'if block'; // SAME 'a' – var is function-scoped!
  let b = 'if-let';    // NEW 'b' – block scoped

  console.log(a); // 'if block'
  console.log(b); // 'if-let'
}

console.log(a); // 'if block' (var leaked out of if!)
console.log(b); // 'block' (let stayed in its block)
}

test();

console.log(a); // 'global' (function var didn't leak out)

```

2.3 The 8 Data Types

JavaScript has **7 primitive types** and **1 object type**. Primitives are immutable values stored by value. Objects (including arrays and functions) are stored by reference.

Number	64-bit IEEE 754 floating point. Both integers and decimals. Special values: Infinity, -Infinity, NaN.	42, 3.14, -7, Infinity, NaN, 1e6
String	Sequence of UTF-16 characters. Immutable. Created with quotes or template literals.	'hello', "world", `Hi \${name}`
Boolean	Logical values. Result of comparisons. Used in conditions.	true, false
Null	Intentional absence of value. A deliberate 'nothing'. typeof null === 'object' (historical bug).	null
Undefined	Variable declared but not assigned. Default return value of functions. Unintentional absence.	undefined
Symbol	Unique, immutable identifier. Useful for unique object keys that won't clash.	Symbol('id')
BigInt	Arbitrary precision integers. For numbers beyond Number.MAX_SAFE_INTEGER ($2^{53} - 1$).	9007199254740992n, BigInt(42)
Object	Collection of key-value pairs. Arrays, functions, dates are all objects. Stored by reference.	{a:1}, [1,2,3], function(){}

2.4 typeof Operator

```

typeof 42          // 'number'
typeof 'hello'     // 'string'

```

```
typeof true          // 'boolean'
typeof undefined     // 'undefined'
typeof null          // 'object'    ← BUG! (legacy)
typeof {}            // 'object'
typeof []            // 'object'    ← arrays are objects
typeof function(){} // 'function' ← special case
typeof Symbol()      // 'symbol'
typeof 42n           // 'bigint'

// To check null: use strict equality
null === null        // true (only reliable way)

// To check array: use Array.isArray()
Array.isArray([])     // true
Array.isArray({})     // false
```

COMMON MISTAKES

- ✗ Using `var` instead of `let/const` — leads to scope confusion and bugs
- ✗ Expecting `typeof null` to return `'null'` — it returns `'object'`
- ✗ Using `==` to check `null` (`null == undefined` is `true` — use `===`)
- ✗ Mutating a `const` object and expecting an error — `const` only prevents reassignment
- ✗ Confusing `undefined` (not assigned) with `null` (deliberately empty)

BEST PRACTICES

- ✓ Default to `const` — only use `let` when you know you need to reassign
- ✓ Never use `var` in modern JavaScript
- ✓ Use `Array.isArray()` to check for arrays, not `typeof`
- ✓ Use `=== null` to check for `null` specifically
- ✓ Use `Number.isNaN()` not `isNaN()` — global `isNaN` coerces first

2.5 Interview Questions

Q: What is the difference between `var`, `let`, and `const`?

`var`: function-scoped, hoisted as `undefined`, can be redeclared, attaches to global object. `let`: block-scoped, hoisted to TDZ, cannot be redeclared. `const`: block-scoped, hoisted to TDZ, cannot be reassigned (but object contents can be mutated). Use `const` by default, `let` for mutation, never `var`.

Q: Why does `typeof null` return `'object'`?

It's a historical bug from JavaScript's first implementation (1995). Values were stored as type tags + value. The `null` type tag was 0, same as objects. The bug was never fixed to preserve backwards compatibility. Always check for `null` with `=== null`.

CHAPTER SUMMARY

- ✓ `const` by default, `let` for mutable values, never `var`
- ✓ 7 primitives: `number`, `string`, `boolean`, `null`, `undefined`, `symbol`, `bigint`
- ✓ 1 object type: includes arrays, functions, objects, dates
- ✓ Primitives are copied by value; objects by reference
- ✓ `typeof null === 'object'` is a known historical bug

PRACTICE QUESTIONS

1. What will `console.log(typeof null)` output and why?
2. Declare a `const` array and add an item to it. Does it throw an error?
3. What is the difference between `null` and `undefined`?
4. What is `Number.MAX_SAFE_INTEGER` and why does it exist?

CHAPTER
03

Operators

Every operator in JavaScript — arithmetic, comparison, logical, assignment, and the modern additions.

3.1 Arithmetic Operators

```
5 + 3    // 8      - addition
5 - 3    // 2      - subtraction
5 * 3    // 15     - multiplication
5 / 3    // 1.666 - division (always float)
5 % 3    // 2      - modulo (remainder)
5 ** 3   // 125    - exponentiation (ES2016)

// Increment / Decrement
let a = 5;
a++;     // post-increment: use then increment
++a;     // pre-increment: increment then use

let x = 5;
console.log(x++); // 5 (uses x, then increments to 6)
console.log(++x); // 7 (increments to 7, then uses)

// Unary + for number conversion
+'42'    // 42 (string to number)
+true    // 1
+false   // 0
+null    // 0
+undefined // NaN
```

3.2 Comparison Operators

```
// Strict equality (===) – no type coercion
5 === 5      // true
5 === '5'    // false (different types)

// Loose equality (==) – type coercion happens
5 == '5'     // true (string coerced to number)
null == undefined // true (special rule)
0 == false   // true (false coerced to 0)
'' == false  // true (both coerce to 0)

// Relational
5 > 3        // true
```

```
5 >= 5    // true
3 < 5     // true

// String comparison (lexicographic)
'banana' > 'apple' // true (b > a in char code)
'10' > '9'        // false! ('1' < '9' as strings)
10 > 9           // true (use numbers, not strings)
```

3.3 Logical Operators

```
// && (AND) – returns first falsy or last value
true && 'hello' // 'hello' (both truthy, returns last)
false && 'hello' // false (short-circuits at first falsy)
null && doWork() // null (doWork never called!)

// || (OR) – returns first truthy or last value
false || 'default' // 'default'
null || 'fallback' // 'fallback'
0 || 42            // 42 (0 is falsy)
'value' || 'other' // 'value' (short-circuits)

// ?? (Nullish Coalescing – ES2020) – only null/undefined
null ?? 'default' // 'default'
undefined ?? 'default' // 'default'
0 ?? 'default' // 0 (0 is NOT nullish!)
'' ?? 'default' // '' (empty string is NOT nullish!)

// ! (NOT)
!true // false
!0 // true (0 is falsy)
!!value // double negation – converts to boolean
```

3.4 Assignment Operators

```
let x = 10;
x += 5; // x = x + 5 → 15
x -= 3; // x = x - 3 → 12
x *= 2; // x = x * 2 → 24
x /= 4; // x = x / 4 → 6
x %= 4; // x = x % 4 → 2
x **= 3; // x = x ** 3 → 8

// Logical assignment (ES2021)
x ||= 5; // x = x || 5 (assign if x is falsy)
x &&= 5; // x = x && 5 (assign if x is truthy)
x ??= 5; // x = x ?? 5 (assign if x is null/undefined)
```

3.5 Ternary & Optional Chaining

```
// Ternary operator: condition ? ifTrue : ifFalse
const age = 20;
const status = age >= 18 ? 'adult' : 'minor'; // 'adult'

// Nest carefully (readability matters)
const grade = score >= 90 ? 'A' : score >= 80 ? 'B' : 'C';

// Optional chaining (?.) – ES2020
const user = { address: { city: 'London' } };
user?.address?.city // 'London'
user?.phone?.number // undefined (no throw!)

// Without optional chaining (old way):
user && user.address && user.address.city

// Optional chaining with methods and arrays
user?.getName?.() // calls if exists, else undefined
arr?.[0] // safe array access
```

COMMON MISTAKES

- ✗ Using `==` instead of `===` (causes unexpected type coercion)
- ✗ Confusing `??` with `||` — `??` only triggers on null/undefined, not all falsy values
- ✗ Not knowing `&&` and `||` return the actual value, not just true/false
- ✗ Comparing strings with `>` `<` expecting numeric order

3.6 Interview Questions

Q: What is the difference between `==` and `===`?

`===` (strict equality) checks value AND type without coercion. `==` (loose equality) performs type coercion before comparing. `'5' == 5` is true but `'5' === 5` is false. Always use `===` to avoid subtle bugs.

Q: What is the difference between `||` and `??`?

`||` returns first truthy value — treats 0, "", false, null, undefined as falsy. `??` returns first non-nullish value — only treats null and undefined specially, preserving 0 and empty string. Use `??` when 0 or "" are valid values.

CHAPTER SUMMARY

- ✓ Always use `===` for comparisons, never `==`
- ✓ `&&` returns first falsy value; `||` returns first truthy value
- ✓ `??` (nullish coalescing) only triggers on null/undefined, not all falsy
- ✓ `?.` (optional chaining) safely accesses nested properties
- ✓ `++` prefix: increment then use; postfix: use then increment

PRACTICE QUESTIONS

1. What does `0 || 'hello'` evaluate to? What about `0 ?? 'hello'`?
2. What is the result of `null == undefined` vs `null === undefined`?
3. What does `!!null` evaluate to and why?

CHAPTER
04

Type Conversion & Coercion

The most surprising JavaScript behaviours — implicit coercion rules and explicit conversion methods.

4.1 Explicit Conversion

```
// To Number  
Number('42')      // 42  
Number('')        // 0  
Number(null)      // 0  
Number(undefined) // NaN  
Number(true)      // 1  
Number(false)     // 0  
Number('abc')     // NaN  
parseInt('42px')   // 42 (stops at non-digit)  
parseFloat('3.14abc') // 3.14
```

```
// To String  
String(42)        // '42'  
String(null)      // 'null'  
String(undefined) // 'undefined'  
String(true)      // 'true'  
(42).toString()   // '42'  
(255).toString(16) // 'ff' (hex)
```

```
// To Boolean  
Boolean(0)        // false  
Boolean('')       // false  
Boolean(null)     // false  
Boolean(undefined) // false  
Boolean(NaN)      // false  
Boolean(false)    // false  
  
// EVERYTHING ELSE is true:  
Boolean('0')      // true (non-empty string!)  
Boolean([])       // true (empty array!)  
Boolean({})       // true (empty object!)
```

4.2 Implicit Coercion — The Famous Traps

```
// + operator: prefers string concatenation  
'5' + 3      // '53' (number coerced to string)
```

```

'5' + true      // '5true'
1 + '2' + 3     // '123'  (left-to-right, 1+'2' = '12')
1 + 2 + '3'     // '33'   (1+2=3, 3+'3'='33')

// Arithmetic (non-+) operators: force number
'5' - 3         // 2      (string coerced to number)
'5' * '2'       // 10
'10' / '2'      // 5
true + true     // 2      (both coerced to 1)
[] + []         // ''     (both convert to empty string)
[] + {}         // '[object Object]'
{} + []         // 0      ({ } parsed as empty block, +[] = 0)

// Comparison coercion
null > 0        // false
null == 0       // false
null >= 0       // true   (!!inconsistent!!)
undefined == null // true  (special rule)
NaN == NaN     // false  (NaN is not equal to itself!)

```

Abstract Equality Coercion Rules (==)

```

Type Coercion Decision Tree for ==
x == y
  ■■■ Same type? → use ===
  ■■■ null == undefined? → true (always)
  ■■■ number == string? → ToNumber(string) then compare
  ■■■ boolean == anything? → ToNumber(boolean) then compare
  ■■■ object == primitive? → ToPrimitive(object) then compare
  ■■■ otherwise → false

```

4.3 The Falsy Six (and One More)

```

// The 7 falsy values – everything else is truthy
if (!false) console.log('false is falsy');
if (!0) console.log('0 is falsy');
if (!-0) console.log('-0 is falsy');
if (!0n) console.log('0n (BigInt) is falsy');
if (!'') console.log('empty string is falsy');
if (!null) console.log('null is falsy');
if (!undefined) console.log('undefined is falsy');
if (!NaN) console.log('NaN is falsy');

// Common traps – these are TRUTHY:
if ('0') console.log('truthy!'); // non-empty string
if ([]) console.log('truthy!'); // empty array
if ({}) console.log('truthy!'); // empty object
if (-1) console.log('truthy!'); // non-zero number

```

COMMON MISTAKES

- ✗ Using `+` with mixed strings and numbers — concatenation, not addition
- ✗ Relying on `null >= 0` being true (null comparison is inconsistent)
- ✗ Using `isNaN()` instead of `Number.isNaN()` — `isNaN('hello')` returns true
- ✗ Expecting `[] == false` to be false (it's true due to coercion: `[]=>"=>0`, `false=>0`)
- ✗ Thinking `0 == false` is fine to use — it works but leads to confusion

4.4 Interview Questions

Q: What is the output of `[] + {}` and `{ } + []`?

`[] + {}` = `'[object Object]'`: `[]` converts to `"` (empty string), `{ }` converts to `'[object Object]'`, string concat. `{ } + []` = `0`: when `{ }` is at the start of a statement, JS parses it as an empty block (not object literal). Then `+` is the unary `+` applied to `[]`, which converts `[]` to `"` to `0`.

Q: Why is `NaN !== NaN`?

`NaN` (Not a Number) is the only value in JavaScript not equal to itself. This is by IEEE 754 specification. The rationale: `NaN` represents the result of invalid operations — there's no single 'not a number' value, so all `NaN` values are considered different. Use `Number.isNaN()` to check for `NaN`.

Q: What is the difference between null and undefined in type coercion?

`null` and `undefined` are loosely equal (`null == undefined` is true) but both are strictly unequal to any other value. In numeric context: `Number(null)` = `0` but `Number(undefined)` = `NaN`. This means `null >= 0` is true (null coerces to 0) but `null == 0` is false (null only equals null/undefined).

CHAPTER SUMMARY

- ✓ Explicit conversion: `Number()`, `String()`, `Boolean()` — clear and predictable
- ✓ `+` operator triggers string concatenation; `-`, `*`, `/` force numbers
- ✓ 7 falsy values: `false`, `0`, `-0`, `0n`, `"`, `null`, `undefined`, `NaN`
- ✓ `[]` and `{ }` are truthy despite being 'empty'
- ✓ Use `Number.isNaN()` not `isNaN()`; use `===` not `==`

PRACTICE QUESTIONS

1. What does `1 + '2' + 3` evaluate to and why?
2. Is `[] == false` true or false? Trace the coercion steps.
3. What does `Number(null)` return? What about `Number(undefined)`?
4. Write a function that safely checks if a value is `NaN`.

CHAPTER
05

Control Flow

if/else, switch, loops — controlling the execution path of your programs.

5.1 if / else if / else

```
const score = 75;

if (score >= 90) {
  console.log('A');
} else if (score >= 80) {
  console.log('B');
} else if (score >= 70) {
  console.log('C');
} else {
  console.log('F');
}

// Output: 'C'

// Truthy/falsy in conditions
const name = '';
if (name) {           // '' is falsy
  console.log('Has name');
} else {
  console.log('No name'); // this runs
}
```

5.2 switch

```
const day = 'Monday';

switch (day) {
  case 'Monday':
  case 'Tuesday':    // fall-through: both match this block
    console.log('Weekday');
    break;           // MUST have break to prevent fall-through!
  case 'Saturday':
  case 'Sunday':
    console.log('Weekend');
    break;
  default:
    console.log('Unknown');
```

```
}  
  
// switch uses strict equality (===)  
// switch (0) { case false: } – will NOT match (0 !== false)
```

5.3 for Loop

```
// Classic for loop  
for (let i = 0; i < 5; i++) {  
  console.log(i); // 0, 1, 2, 3, 4  
}  
  
// for...of – iterate over iterable values (arrays, strings, maps)  
const fruits = ['apple', 'banana', 'cherry'];  
for (const fruit of fruits) {  
  console.log(fruit);  
}  
  
// for...in – iterate over object KEYS (enumerable properties)  
const obj = { a: 1, b: 2, c: 3 };  
for (const key in obj) {  
  console.log(key, obj[key]); // 'a' 1, 'b' 2, 'c' 3  
}  
  
// WARNING: for...in also iterates prototype properties!  
// Use hasOwnProperty() check or Object.keys() instead
```

5.4 while & do-while

```
// while – checks condition BEFORE each iteration  
let count = 0;  
while (count < 3) {  
  console.log(count); // 0, 1, 2  
  count++;  
}  
  
// do-while – executes AT LEAST ONCE, checks AFTER  
let n = 10;  
do {  
  console.log(n); // prints 10 even though 10 >= 5  
  n--;  
} while (n < 5);
```

5.5 break & continue

```
// break – exits the loop entirely  
for (let i = 0; i < 10; i++) {
```

```
    if (i === 5) break;

    console.log(i); // 0, 1, 2, 3, 4
  }

// continue – skips rest of current iteration
for (let i = 0; i < 5; i++) {
  if (i === 2) continue;
  console.log(i); // 0, 1, 3, 4 (skips 2)
}

// Labelled break – break out of nested loops
outer: for (let i = 0; i < 3; i++) {
  for (let j = 0; j < 3; j++) {
    if (j === 1) break outer; // breaks OUTER loop
    console.log(i, j); // only logs: 0 0
  }
}
```

COMMON MISTAKES

- ✗ Forgetting break in switch cases — causes unintended fall-through
- ✗ Using for...in on arrays — iterates indices as strings + prototype props
- ✗ Infinite loop: while(true) without a break condition
- ✗ Modifying the loop array during forEach/for...of iteration

5.6 Interview Questions

Q: What is the difference between for...in and for...of?

for...in iterates over enumerable property KEYS of an object (including inherited ones). for...of iterates over VALUES of any iterable (arrays, strings, Maps, Sets, generators). Don't use for...in on arrays — it iterates indices as strings and may include prototype properties.

Q: When would you use do-while instead of while?

When you need the loop body to execute at least once regardless of the condition. Common use case: prompting a user for valid input — you always show the prompt once, then keep prompting if the input is invalid.

CHAPTER SUMMARY

- ✓ Use === in all comparisons — switch also uses strict equality
- ✓ Always add break in switch cases unless intentional fall-through
- ✓ for...of for array values; for...in for object keys (with caution)
- ✓ while checks before iteration; do-while always runs at least once
- ✓ break exits loop; continue skips to next iteration

PRACTICE QUESTIONS

1. Write a loop that prints all even numbers from 1 to 20.
2. Use for...of to sum all values in an array.
3. Rewrite a for loop as a while loop.
4. What is a labelled break and when would you use it?

CHAPTER
06

Functions

First-class citizens of JavaScript — the building blocks of all programs, explained from basics to internals.

6.1 Function Declaration

A function declaration creates a named function using the **function** keyword. It is **fully hoisted** — you can call it before it appears in the code.

```
// Can be called BEFORE the declaration (hoisted)
greet('Alice'); // 'Hello, Alice!'

function greet(name) {
  return `Hello, ${name}!`;
}

// Default parameters (ES6)
function add(a, b = 0) {
  return a + b;
}

add(5); // 5 (b defaults to 0)
add(5, 3); // 8

// Rest parameters – collects remaining args into array
function sum(...numbers) {
  return numbers.reduce((total, n) => total + n, 0);
}

sum(1, 2, 3, 4); // 10
```

6.2 Function Expression

A function expression assigns a function to a variable. It is **NOT hoisted** — only the variable declaration is hoisted, not the function value.

```
// sayHi is undefined here if we try to call it (var)
// TypeError if let/const (TDZ)

const multiply = function(x, y) {
  return x * y;
};

multiply(4, 5); // 20

// Named function expression (useful for recursion/debugging)
const factorial = function fact(n) {
  return n <= 1 ? 1 : n * fact(n - 1); // can use 'fact' internally
}
```

```
};  
factorial(5); // 120
```

6.3 Arrow Functions

Arrow functions (`=>`) are a concise syntax introduced in ES6. Key difference: they do **NOT** have their own `this`, `arguments`, or `prototype`.

```
// Concise syntax  
const double = x => x * 2; // single param, implicit return  
const add = (a, b) => a + b; // multiple params  
const greet = () => 'Hello!'; // no params  
  
// Block body (needs explicit return)  
const square = x => {  
  const result = x * x;  
  return result;  
};  
  
// Returning an object literal (wrap in parens!)  
const makeUser = name => ({ name, role: 'user' });  
// Without parens: name => { name } is a block, not object!  
  
// No 'arguments' object  
const fn = (...args) => args; // use rest params instead
```

6.4 IIFE — Immediately Invoked Function Expression

```
// IIFE: define and immediately call  
(function() {  
  const secret = 'private'; // not accessible outside  
  console.log('IIFE runs immediately');  
})();  
  
// Arrow IIFE  
(() => {  
  console.log('Arrow IIFE');  
})();  
  
// IIFE with parameters  
(function(global) {  
  // 'global' is window in browser, global in Node  
})(globalThis);  
  
// Modern use: async IIFE for top-level await  
(async () => {  
  const data = await fetchData();  
  console.log(data);  
})
```

```
})();
```

6.5 First-Class Functions

In JavaScript, functions are **first-class citizens** — they can be assigned to variables, passed as arguments, returned from other functions, and stored in data structures. This enables higher-order functions and functional programming patterns.

```
// Functions as arguments (callback)
```

```
function runTwice(fn) {
```

```
  fn();
```

```
  fn();
```

```
}
```

```
runTwice(() => console.log('Hi')); // Hi Hi
```

```
// Functions as return values
```

```
function makeAdder(x) {
```

```
  return function(y) { return x + y; }; // closure!
```

```
}
```

```
const add5 = makeAdder(5);
```

```
add5(3); // 8
```

```
add5(7); // 12
```

```
// Functions in arrays / objects
```

```
const ops = [Math.sqrt, Math.abs, Math.ceil];
```

```
ops.forEach(fn => console.log(fn(4.2)));
```

COMMON MISTAKES

- ✗ Forgetting () when returning an object literal from arrow function
- ✗ Using arrow function as object method (loses correct this binding)
- ✗ Calling a function expression before its declaration (TypeError/TDZ)
- ✗ Missing return statement in multi-line arrow function body

BEST PRACTICES

- ✓ Use arrow functions for callbacks and short expressions
- ✓ Use function declarations for named, reusable functions
- ✓ Use default parameters instead of checking arguments inside the function
- ✓ Use rest parameters (...args) instead of the arguments object

6.6 Interview Questions

Q: What is the difference between function declaration and function expression?

Function declaration: `function foo() {}` — fully hoisted (can call before). Function expression: `const foo = function() {}` — only variable hoisted, not function (undefined/TDZ). Arrow: `const foo = () => {}` — expression with

no own this/arguments. Declarations are good for utility functions; expressions for passing around or conditional creation.

Q: What is a higher-order function?

A function that takes another function as an argument, returns a function, or both. Examples: Array.map/filter/reduce take callback functions. setTimeout/setInterval take callback functions. Functions like memoize, debounce, and curry return functions. This is possible because functions are first-class objects in JavaScript.

CHAPTER SUMMARY

- ✓ Function declarations are hoisted (can call before definition)
- ✓ Function expressions are not hoisted (variable is undefined/TDZ)
- ✓ Arrow functions have no own this, arguments, or prototype
- ✓ IIFE creates an isolated scope; useful for avoiding global pollution
- ✓ Functions are first-class: assign, pass, return like any value

PRACTICE QUESTIONS

1. What is the difference between foo() and foo as a value?
2. Write a function factory that creates power functions (square, cube, etc.)
3. Why can't you use an arrow function as a constructor with new?
4. What does the arguments object contain and when is it unavailable?

CHAPTER
07

Scope

Understanding where variables live — global, function, and block scope with the scope chain.

7.1 What Is Scope?

Scope defines the accessibility of variables — where in your code a variable can be read or written. JavaScript uses **lexical scoping**: scope is determined by where code is written, not where it is executed.

7.2 Global Scope

```
// Variables declared outside any function/block
var globalVar = 'I am global';
let globalLet = 'Also global';

function test() {
  console.log(globalVar); // accessible
  console.log(globalLet); // accessible
}

// var at global level attaches to window object (browser)
var name = 'Alice';
console.log(window.name); // 'Alice' (browser only)

// let/const do NOT attach to window
let age = 25;
console.log(window.age); // undefined
```

7.3 Function Scope

```
function outer() {
  var functionVar = 'in function';
  let functionLet = 'in function too';

  console.log(functionVar); // accessible
}

// console.log(functionVar); // ReferenceError – outside scope

// var is function-scoped – leaks from blocks!
function test() {
  if (true) {
    var leaked = 'I leaked!'; // scoped to test(), NOT the if block
    let blocked = 'I stayed'; // scoped to the if block
```

```

}
console.log(leaked); // 'I leaked!'
// console.log(blocked); // ReferenceError
}

```

7.4 Block Scope

```

// let and const are block-scoped
{
  let blockVar = 'only here';
  const PI = 3.14;
}
// blockVar and PI not accessible here

// Block scope in loops
for (let i = 0; i < 3; i++) {
  // Each iteration gets its OWN i (new binding per iteration)
  setTimeout(() => console.log(i), 100); // 0, 1, 2
}

for (var j = 0; j < 3; j++) {
  // All iterations SHARE the same j (function-scoped)
  setTimeout(() => console.log(j), 100); // 3, 3, 3
}

```

7.5 Scope Chain

Scope Chain Diagram

```

function outer() {
  let x = 10;
  function middle() {
    let y = 20;
    function inner() {
      let z = 30;
      console.log(x + y + z); // 60 - walks scope chain
    }
    inner();
  }
  middle();
}

```

Scope chain for inner():

```

inner scope { z:30 }
  → middle scope { y:20 }
    → outer scope { x:10 }
      → global scope { ... }
        → null (end of chain)

```

Variable lookup goes UP the chain until found (or ReferenceError)

7.6 Interview Questions

Q: What is the scope chain?

When a variable is accessed, JavaScript looks in the current scope first. If not found, it moves to the outer scope, then outer's outer, up to the global scope. If not found at global scope, `ReferenceError` is thrown. This chain of scope environments is the scope chain. It's determined at definition time (lexical scoping), not call time.

Q: Why does `var` leak out of `if` blocks but not function bodies?

`var` is function-scoped, not block-scoped. Blocks (`if`, `for`, `while`, `{}`) do not create a new scope for `var` — only functions do. This is a design flaw in early JavaScript. `let` and `const` are block-scoped and solve this problem.

CHAPTER SUMMARY

- ✓ Scope controls variable accessibility
- ✓ Global scope: accessible everywhere; avoid polluting it
- ✓ Function scope: `var` is contained in the function
- ✓ Block scope: `let/const` are contained in the `{}` block
- ✓ Scope chain: lookup travels up from inner to outer to global

PRACTICE QUESTIONS

1. What is the output of the `var` loop vs `let` loop `setTimeout` example?
2. Can a function access variables declared after it in the code?
3. What is the difference between scope and context (`this`)?

CHAPTER
08

Hoisting

How the JavaScript engine moves declarations to the top of their scope — and what really happens internally.

8.1 What Is Hoisting?

During the **creation phase** of an Execution Context, the JavaScript engine scans through the code and allocates memory for all variable and function declarations before execution begins. The declarations are conceptually 'moved to the top' of their scope — this is called hoisting.

Important: Only declarations are hoisted, NOT initializations. The value is not moved — only the existence of the variable is registered.

8.2 Variable Hoisting

```
// What you write:
console.log(x); // undefined (not ReferenceError!)
var x = 5;
console.log(x); // 5

// How the engine sees it (after hoisting):
var x;           // declaration hoisted (value = undefined)
console.log(x);  // undefined
x = 5;           // assignment stays in place
console.log(x);  // 5

// let and const – HOISTED but in Temporal Dead Zone
// console.log(y); // ReferenceError: Cannot access 'y' before init
let y = 10;
console.log(y);  // 10
```

8.3 Function Hoisting

```
// Function DECLARATIONS are fully hoisted (with body)
greet(); // 'Hello!' – works before declaration

function greet() {
  console.log('Hello!');
}

// Function EXPRESSIONS are NOT fully hoisted
// console.log(sayHi); // undefined (var) or TDZ error (let)
// sayHi();           // TypeError: sayHi is not a function

var sayHi = function() { // only 'var sayHi' is hoisted
```

```

console.log('Hi!');

};

// Priority: function declarations win over var
console.log(typeof foo); // 'function' (not 'undefined')
var foo = 'string';
function foo() {}
console.log(typeof foo); // 'string' (assignment happens after)

```

8.4 Temporal Dead Zone (TDZ)

The **Temporal Dead Zone** is the period between the start of a scope and the point where a `let/const` variable is initialised. Accessing the variable during this period throws a **ReferenceError**. The TDZ exists to catch use-before-declare bugs that `var` silently allowed.

Temporal Dead Zone Visualisation

```

■ Function scope begins
■ let x; <- x ENTERS TDZ here (hoisted but not initialised)
■ TEMPORAL DEAD ZONE for x
■ console.log(x); <- ReferenceError thrown here!
■ let x = 10; <- x EXITS TDZ (initialised with 10)
■ console.log(x); // 10 <- safe to use now

```

8.5 Hoisting Order of Priority

```

// Order: function declarations > var declarations

// 1. Function declarations processed first
function example() { return 'function'; }

var example = 'variable';

// How engine processes this:
// Step 1: function example() registered (full body)
// Step 2: var example – already declared, skip
// Step 3: var example = 'variable' – assignment runs

console.log(example); // 'variable' (assignment happened last)

// Multiple function declarations – LAST ONE wins
function talk() { return 'first'; }
function talk() { return 'second'; }
talk(); // 'second'

```

COMMON MISTAKES

- ✗ Relying on var hoisting — use let/const and declare before use
- ✗ Thinking let/const are not hoisted — they ARE but land in TDZ
- ✗ Confusing function declaration hoisting with expression hoisting
- ✗ Using a variable inside a function that shadows a global — confusing behavior

8.6 Interview Questions

Q: What is hoisting and how does it differ for var vs let/const?

Hoisting is JS engine moving declarations to top of scope during creation phase. var: hoisted AND initialised to undefined — accessible before declaration (value is undefined). let/const: hoisted but NOT initialised — placed in TDZ. Accessing before declaration throws ReferenceError. Function declarations: fully hoisted with body.

Q: What is the Temporal Dead Zone?

The TDZ is the period from the start of a block scope to the point where a let/const variable declaration is reached. During this period, the variable exists in the scope (hoisted) but accessing it throws ReferenceError. This prevents the confusing pattern var allowed where a variable could be accessed as undefined before its declaration.

CHAPTER SUMMARY

- ✓ Hoisting moves declarations (not values) to top of scope during creation phase
- ✓ var: hoisted + initialised to undefined — accessible (but undefined) before declaration
- ✓ let/const: hoisted but in TDZ — ReferenceError if accessed before declaration
- ✓ Function declarations: fully hoisted (body included) — can call before definition
- ✓ Function expressions: only the variable is hoisted, not the function

PRACTICE QUESTIONS

1. What is the output: `console.log(a); var a = 5; console.log(a);`
2. What happens when you access a let variable before its declaration?
3. Does function hoisting override var declaration? Show an example.
4. Why is it recommended to declare variables at the top of their scope?

CHAPTER
09

Arrays

The most-used data structure in JavaScript — every important method with examples and interview tips.

9.1 Creating Arrays

```
const arr1 = [1, 2, 3];           // literal (preferred)
const arr2 = new Array(3);        // [empty × 3]
const arr3 = Array.from('abc');   // ['a', 'b', 'c']
const arr4 = Array.from({length:3}, (_,i) => i*2); // [0,2,4]
const arr5 = Array.of(1, 2, 3);   // [1,2,3]
const arr6 = [...arr1, ...arr3];   // spread: [1,2,3,'a','b','c']
```

map(callback)

Creates a NEW array by transforming each element. Does not mutate original.

```
const nums = [1, 2, 3, 4];
const doubled = nums.map(n => n * 2); // [2, 4, 6, 8]
const users = [{name:'Alice'}, {name:'Bob'}];
const names = users.map(u => u.name); // ['Alice', 'Bob']
```

filter(callback)

Creates a NEW array with only elements where callback returns true.

```
const nums = [1, 2, 3, 4, 5, 6];
const evens = nums.filter(n => n % 2 === 0); // [2, 4, 6]
const adults = users.filter(u => u.age >= 18);
```

reduce(callback, initial)

Reduces array to a single value by accumulating. Most versatile method.

```
const sum = [1,2,3,4,5].reduce((acc, n) => acc + n, 0); // 15
// Build object from array
const byId = users.reduce((map, u) => {
  map[u.id] = u;
  return map;
}, {});
// Count occurrences
const freq = ['a','b','a','c','b','a'].reduce((acc, ch) => {
  acc[ch] = (acc[ch] || 0) + 1;
  return acc;
}, {}); // {a:3, b:2, c:1}
```

find(callback)

Returns the **FIRST** element matching the condition, or undefined.

```
const users = [{id:1,name:'Alice'}, {id:2,name:'Bob'}];  
const alice = users.find(u => u.id === 1); // {id:1,name:'Alice'}  
const nobody = users.find(u => u.id === 99); // undefined
```

findIndex(callback)

Returns the index of the first matching element, or -1.

```
const idx = users.findIndex(u => u.name === 'Bob'); // 1
```

some(callback)

Returns true if **AT LEAST ONE** element passes. Short-circuits.

```
[1,2,3,4].some(n => n > 3) // true  
[1,2,3,4].some(n => n > 10) // false
```

every(callback)

Returns true if **ALL** elements pass. Short-circuits on first false.

```
[2,4,6,8].every(n => n % 2 === 0) // true (all even)  
[2,4,5,8].every(n => n % 2 === 0) // false (5 is odd)
```

sort(compareFn)

Sorts **IN PLACE**. Without compareFn sorts lexicographically (strings). Provide compareFn for numbers!

```
// WRONG for numbers:  
[10, 1, 21].sort() // [1, 10, 21] (lexicographic!)  
  
// CORRECT:  
[10, 1, 21].sort((a,b) => a - b) // [1, 10, 21] ascending  
[10, 1, 21].sort((a,b) => b - a) // [21, 10, 1] descending  
  
// Sort strings (locale-aware):  
['banana','apple','cherry'].sort() // ['apple','banana','cherry']
```

slice(start, end)

Returns a **NEW** array (copy). Does **NOT** mutate. end is exclusive.

```
const arr = [1,2,3,4,5];  
arr.slice(1, 3) // [2, 3]  
arr.slice(-2) // [4, 5] (last 2)  
arr.slice() // [1,2,3,4,5] (full copy)
```

splice(start, deleteCount, ...items)

Mutates the original array. Removes and/or inserts elements. Returns removed elements.

```
const arr = [1,2,3,4,5];  
arr.splice(1, 2) // returns [2,3]; arr is now [1,4,5]  
arr.splice(1, 0, 'a','b') // inserts at index 1; arr: [1,'a','b',4,5]  
arr.splice(2, 1, 'X') // replace 1 element at idx 2
```


flat(depth) & flatMap()

flat() flattens nested arrays. flatMap() maps then flattens one level.

```
[1,[2,[3,[4]]]].flat()    // [1,2,[3,[4]]] (depth 1 default)
[1,[2,[3,[4]]]].flat(Infinity) // [1,2,3,4]
[1,2,3].flatMap(n => [n, n*2]) // [1,2,2,4,3,6]
```

9.2 Other Essential Methods

```
// push / pop – add/remove from END (mutates)
arr.push(6);    // adds 6 to end; returns new length
arr.pop();      // removes last element; returns it

// unshift / shift – add/remove from BEGINNING (mutates)
arr.unshift(0); // adds 0 to start; returns new length
arr.shift();    // removes first element; returns it

// includes – check if value exists
[1,2,3].includes(2); // true
[1,2,3].includes(5); // false

// indexOf / lastIndexOf
[1,2,3,2].indexOf(2);    // 1 (first occurrence)
[1,2,3,2].lastIndexOf(2); // 3 (last occurrence)

// join – array to string
[1,2,3].join('-'); // '1-2-3'
[1,2,3].join('');  // '123'

// forEach – iterate (no return value, cannot break)
[1,2,3].forEach((val, idx) => console.log(idx, val));

// Array spread and concat
[...arr1, ...arr2]    // spread
arr1.concat(arr2, arr3) // concat (returns new array)

// fill – fill with a value
new Array(5).fill(0)  // [0,0,0,0,0]
```

COMMON MISTAKES

- ✗ Using sort() on numbers without a compare function (lexicographic order)
- ✗ Confusing slice (returns copy) with splice (mutates original)
- ✗ Using indexOf() instead of includes() for existence check
- ✗ forEach does not return a value — don't try to chain it like map
- ✗ Sparse arrays from new Array(n) behave unexpectedly with map/filter

9.3 Interview Questions

Q: What is the difference between map, filter, and reduce?

map: transforms each element, returns same-length array. filter: keeps elements matching condition, returns shorter/equal array. reduce: aggregates all elements into a single value (any type). All three return new arrays/values and don't mutate the original.

Q: What is the difference between slice and splice?

slice(start, end): non-mutating, returns a new array containing elements from start to end (exclusive). splice(start, deleteCount, ...items): mutates original array, removes deleteCount elements starting at start, optionally inserts new items, returns array of removed elements.

CHAPTER SUMMARY

- ✓ map/filter/reduce are the most important array methods — master them
- ✓ sort() mutates and sorts lexicographically by default — always provide compareFn for numbers
- ✓ slice is non-mutating; splice mutates — know the difference
- ✓ find returns first match; filter returns ALL matches
- ✓ some = at least one; every = all must pass

PRACTICE QUESTIONS

1. Given [1,2,3,4,5], use reduce to compute the product of all numbers.
2. Filter an array of objects to only include those where active === true.
3. Sort an array of objects by their .name property alphabetically.
4. Use flatMap to double each number in [[1,2],[3,4],[5,6]].

CHAPTER
10

Strings

Everything about strings — methods, template literals, and the questions every interviewer asks.

10.1 Creating Strings

```
const s1 = 'single quotes';
const s2 = "double quotes";
const s3 = `template literal ${1 + 2}`; // '3'

// Template literals: multi-line and expressions
const html = `
  <div>
    <h1>${title}</h1>
  </div>
`;

// String length (character count)
'hello'.length; // 5
''.length;      // 0

// Strings are IMMUTABLE – methods return new strings
let str = 'hello';
str[0] = 'H';    // silently fails – can't mutate
str.toUpperCase(); // 'HELLO' – new string, str unchanged
```

10.2 String Methods Reference

toUpperCase() / toLowerCase()

```
'Hello World'.toUpperCase() // 'HELLO WORLD'
'Hello World'.toLowerCase()  // 'hello world'
```

trim() / trimStart() / trimEnd()

```
' hello '.trim()           // 'hello'
' hello '.trimStart()      // 'hello '
' hello '.trimEnd()        // ' hello'
```

split(separator)

```
'a,b,c'.split(',')         // ['a','b','c']
'hello'.split('')          // ['h','e','l','l','o']
'hello'.split()            // ['hello'] (no split)
```

slice(start, end)

```
'hello world'.slice(6)      // 'world'
'hello world'.slice(0, 5)   // 'hello'
```

```
'hello world'.slice(-5) // 'world' (from end)
```

indexOf(str) / lastIndexOf(str)

```
'hello'.indexOf('l') // 2
```

```
'hello'.lastIndexOf('l') // 3
```

```
'hello'.indexOf('z') // -1 (not found)
```

includes(str)

```
'hello world'.includes('world') // true
```

```
'hello world'.includes('xyz') // false
```

startsWith(str) / endsWith(str)

```
'hello'.startsWith('he') // true
```

```
'hello'.endsWith('lo') // true
```

replace(search, replacement)

```
'hello world'.replace('world', 'JS') // 'hello JS'
```

```
// Only replaces FIRST match for string search
```

```
'aababc'.replace('a', 'X') // 'Xababc' (only first!)
```

```
'aababc'.replaceAll('a', 'X') // 'XXbXbc' (all, ES2021)
```

```
'hello'.replace(/l/g, 'r') // 'herro' (regex global)
```

padStart(length, fill) / padEnd()

```
'5'.padStart(3, '0') // '005' (zero-padding)
```

```
'hi'.padEnd(5, '.') // 'hi...' (padding right)
```

repeat(count)

```
'ha'.repeat(3) // 'hahaha'
```

charAt(index) / charCodeAt(index)

```
'hello'.charAt(1) // 'e'
```

```
'hello'.charCodeAt(1) // 101 (ASCII/Unicode code)
```

String.fromCharCode(code)

```
String.fromCharCode(65) // 'A'
```

match(regex)

```
'test123'.match(/[0-9]+)/ // ['123']
```

```
'test123'.match(/[a-z]+/g) // ['test']
```

at(index) [ES2022]

```
'hello'.at(0) // 'h'
```

```
'hello'.at(-1) // 'o' (negative index from end!)
```

10.3 Template Literals Deep Dive

```
// Tagged templates
```

```
function highlight(strings, ...values) {
```

```
  return strings.reduce((result, str, i) => {
```

```
    return result + str + (values[i] ? `${values[i]}` : '');
```

```
  }, '');
```

```
}
```

```
const name = 'Alice';  
const score = 95;  
highlight`Hello ${name}, your score is ${score}!`;  
// 'Hello [Alice], your score is [95]!'
```

COMMON MISTAKES

- ✗ Using + to concatenate many strings (use template literals)
- ✗ Confusing slice (works on strings AND arrays) with substring (strings only)
- ✗ indexOf returns -1 for not found — always check !== -1 or use includes()
- ✗ replace() without regex /g only replaces the first match
- ✗ Forgetting strings are immutable — methods return new strings

10.4 Interview Questions

Q: How do you reverse a string in JavaScript?

str.split("").reverse().join("") — split into chars array, reverse array, join back. Modern one-liner: [...str].reverse().join("") using spread to handle Unicode correctly. For proper Unicode (emoji): use Array.from(str).reverse().join("").

Q: How do you check if a string is a palindrome?

const isPalindrome = s => s === s.split("").reverse().join(""); Or without reversing: iterate from both ends — compare s[0] with s[length-1], s[1] with s[length-2], etc.

Q: What is the difference between slice and substring?

Both extract a portion of a string. slice: supports negative indices (counts from end); if start > end, returns "". substring: does not support negatives (treats as 0); if start > end, swaps them. Prefer slice as it's more consistent.

CHAPTER SUMMARY

- ✓ Strings are immutable — all methods return new strings
- ✓ Template literals support multi-line, expressions, and tagged templates
- ✓ split/join are inverse operations (string↔array)
- ✓ includes() for existence; indexOf() when you need the position
- ✓ replaceAll() or regex /g flag to replace all occurrences

PRACTICE QUESTIONS

1. Capitalise the first letter of every word in a string.
2. Count the occurrences of a character in a string.
3. Check if two strings are anagrams of each other.
4. Truncate a string to N characters and add '...' if longer.

CHAPTER
11

Objects

The fundamental data structure of JavaScript — creation, manipulation, destructuring, and spread.

11.1 Creating Objects

```
// Object literal (most common)
const user = {
  name: 'Alice',
  age: 30,
  'full name': 'Alice Smith', // quoted key for spaces/special chars
};

// Constructor function
function Person(name, age) {
  this.name = name;
  this.age = age;
}

const p = new Person('Bob', 25);

// ES6 Class (modern)
class Animal { constructor(n) { this.name = n; } }

// Object.create() – inherit from prototype
const proto = { greet() { return `Hi, I'm ${this.name}`; } };
const obj = Object.create(proto);
obj.name = 'Charlie';

// Factory function
function createUser(name, age) { return { name, age }; }
```

11.2 Accessing & Modifying Properties

```
const obj = { name: 'Alice', age: 30 };

// Dot notation
obj.name; // 'Alice'
obj.name = 'Bob'; // modify

// Bracket notation (use for dynamic keys)
obj['age']; // 30
const key = 'name';
obj[key]; // 'Alice' – dynamic key access

// Adding new properties
```

```
obj.email = 'alice@example.com';

// Deleting properties
delete obj.age;    // true; obj no longer has 'age'

// Check property existence
'name' in obj;      // true (own + inherited)
obj.hasOwnProperty('name'); // true (own only)
Object.hasOwn(obj, 'name'); // ES2022 – preferred
```

11.3 Object Methods

```
const obj = { a: 1, b: 2, c: 3 };

Object.keys(obj)    // ['a', 'b', 'c']    – enumerable own keys
Object.values(obj)  // [1, 2, 3]          – enumerable own values
Object.entries(obj) // [['a',1],['b',2],['c',3]] – key-value pairs

// Iterate over entries
for (const [key, val] of Object.entries(obj)) {
  console.log(`${key}: ${val}`);
}

// Object.assign(target, ...sources) – shallow merge
const merged = Object.assign({}, obj, { d: 4 });

// Object.freeze() – make immutable
const frozen = Object.freeze({ x: 1 });
frozen.x = 99; // silently fails (TypeError in strict mode)

// Object.fromEntries() – entries array back to object
Object.fromEntries(['a', 1], ['b', 2]); // {a:1, b:2}
```

11.4 Destructuring

```
const user = { name: 'Alice', age: 30, role: 'admin' };

// Basic destructuring
const { name, age } = user;

// Rename
const { name: userName, age: userAge } = user;

// Default values
const { name, score = 100 } = user; // score = 100 (not in object)

// Rest in destructuring
const { name: n, ...rest } = user; // rest = {age:30, role:'admin'}

// Nested destructuring
const { address: { city } } = { address: { city: 'London' } };
```

```
// Function parameter destructuring
function greet({ name, age = 25 }) {
  return `${name} is ${age}`;
}

greet(user); // 'Alice is 30'

// Array destructuring
const [first, second, ...rest2] = [1, 2, 3, 4, 5];
const [, , third] = [1, 2, 3]; // skip elements with comma

// Swap variables
let a = 1, b = 2;
[a, b] = [b, a]; // a=2, b=1
```

11.5 Spread Operator

```
// Spread for objects – shallow copy / merge
const defaults = { theme: 'dark', lang: 'en' };
const overrides = { lang: 'fr', fontSize: 16 };

const config = { ...defaults, ...overrides };
// { theme: 'dark', lang: 'fr', fontSize: 16 }
// NOTE: later properties overwrite earlier ones

// Shallow copy
const copy = { ...original }; // new object, shallow

// Add/override properties
const updatedUser = { ...user, age: 31, isAdmin: true };

// Spread in function calls
const nums = [3, 1, 4, 1, 5];
Math.max(...nums); // 5

// Combine arrays
const all = [...arr1, ...arr2, 'extra'];
```

11.6 Shorthand Property & Computed Keys

```
const name = 'Alice';
const age = 30;

// Shorthand property (ES6)
const user = { name, age }; // same as { name: name, age: age }

// Shorthand method
const obj = {
  greet() { return 'Hello!'; } // instead of greet: function() {}
};
```



```
// Computed property keys
const prefix = 'user';
const dynamic = {
  [prefix + 'Name']: 'Alice', // key: 'userName'
  [`${prefix}Age`]: 30,      // key: 'userAge'
};
```

COMMON MISTAKES

- ✗ Confusing spread with deep copy — spread only copies one level
- ✗ Using `for...in` on objects without `hasOwnProperty` check
- ✗ Destructuring undefined (e.g., `const {x} = undefined`) — `TypeError`
- ✗ Assuming `Object.keys` order — it's generally insertion order but not guaranteed for all engines

11.7 Interview Questions

Q: What is the difference between `Object.assign()` and spread operator?

Both do a shallow merge. `Object.assign(target, source)` modifies `target` in place and returns it. `Spread ({...source})` always creates a new object. They behave identically for merging: `{...a, ...b} === Object.assign({}, a, b)`. Spread is generally preferred as it's more readable and doesn't mutate.

Q: How do you deep clone an object?

1) `structuredClone(obj)` — modern native deep clone (Node 17+/browsers). 2) `JSON.parse(JSON.stringify(obj))` — works for JSON-compatible data (no functions, no dates, no circular refs). 3) `Lodash _.cloneDeep(obj)` — handles edge cases. Spread and `Object.assign` are SHALLOW — nested objects remain as shared references.

CHAPTER SUMMARY

- ✓ Objects store key-value pairs; keys are strings or Symbols
- ✓ Use `Object.keys/values/entries` for iteration
- ✓ Destructuring extracts values cleanly from objects and arrays
- ✓ Spread operator creates shallow copies and merges
- ✓ `Object.freeze` prevents modification; `Object.seal` prevents add/delete

PRACTICE QUESTIONS

1. Merge two objects with conflicting keys — which value wins with spread?
2. Write a function that picks specific keys from an object.
3. Convert an array of `[key, value]` pairs to an object.
4. Use destructuring to swap two variables without a temp variable.

**CHAPTER
12**

Interview Questions

100 Beginner Questions + 75 Intermediate Questions with complete, detailed answers.

Section A: 100 Beginner Questions

Q1. What is JavaScript?

JavaScript is a high-level, interpreted, dynamically-typed, single-threaded programming language. It runs in browsers natively and on servers via Node.js. It's the core language of the web, enabling interactive UI and server-side applications.

Q2. What is the difference between `let`, `var`, and `const`?

`var`: function-scoped, hoisted as undefined, can be redeclared. `let`: block-scoped, hoisted in TDZ, cannot be redeclared. `const`: block-scoped, hoisted in TDZ, cannot be reassigned (but object properties can change).

Q3. What are the primitive data types in JavaScript?

7 primitives: number, string, boolean, null, undefined, symbol, bigint. Everything else (objects, arrays, functions) is of type object.

Q4. What is the difference between `null` and `undefined`?

`undefined`: declared but not assigned; default return of functions; missing function arguments. `null`: explicitly set to 'no value'. Both are falsy. `null == undefined` is true; `null === undefined` is false.

Q5. What does `typeof` operator return?

Returns a string: 'number', 'string', 'boolean', 'undefined', 'symbol', 'bigint', 'function', 'object'. Notable: `typeof null` is 'object' (bug), `typeof function` is 'function' (special case).

Q6. What is NaN?

Not a Number — result of invalid math operations like `0/0` or `parseInt('abc')`. `typeof NaN` is 'number'. It's the only value not equal to itself. Check with `Number.isNaN()`.

Q7. What is the difference between `==` and `===`?

`==` uses type coercion before comparing. `===` checks value AND type without coercion. Always prefer `===` to avoid unexpected behavior.

Q8. What is a template literal?

String delimited by backticks (``) that supports embedded expressions `${expr}`, multi-line strings, and tagged templates. More readable than string concatenation.

Q9. What are falsy values in JavaScript?

false, 0, -0, 0n, "", null, undefined, NaN. All other values are truthy (including '0', [], {}).

Q10. What is array destructuring?

Syntax to unpack array values into variables: `const [a, b] = [1, 2]`. Supports defaults, skipping, rest: `const [first, , third, ...rest] = arr`.

Q11. What is object destructuring?

Syntax to extract object properties into variables: `const {name, age} = user`. Supports renaming (`{name: userName}`), defaults (`{score=0}`), rest (`{x, ...remaining}`).

Q12. What does the spread operator do?

... spreads elements of an iterable. For arrays: [...arr1, ...arr2] merges. For objects: {...obj1, ...obj2} shallow merges. For function calls: fn(...args) spreads args.

Q13. What is the difference between push and pop?

push(...items): adds items to END of array, returns new length. pop(): removes last item, returns it.

Q14. What is the difference between shift and unshift?

unshift(...items): adds to START, returns new length. shift(): removes first element, returns it.

Q15. What does slice return?

A new array containing elements from start to end (exclusive). Does NOT mutate original. Supports negative indices.

Q16. What does splice do?

Mutates array in place. splice(start, deleteCount, ...items). Removes deleteCount elements, optionally inserts new ones. Returns array of removed elements.

Q17. What is the difference between map and forEach?

map: returns a new array of transformed values. forEach: returns undefined, used for side effects only. You can chain map; you cannot chain forEach.

Q18. How does filter work?

Creates a new array containing only elements for which the callback returns truthy. Does not mutate original. Result may be shorter than source.

Q19. What does reduce do?

Accumulates array elements into a single value. Takes callback(accumulator, currentValue) and optional initial value. Can produce any type: number, string, object, array.

Q20. How do you check if a value is an array?

Array.isArray(value) — the only reliable way. typeof [] returns 'object', not 'array'. instanceof Array can fail across iframes.

Q21. What is a callback function?

A function passed as an argument to another function, to be called at a later time. Examples: setTimeout(callback, delay), arr.map(callback), event listeners.

Q22. What is an arrow function?

Concise function syntax using =>. No own this, arguments, super, or prototype. Cannot be used as constructors. Great for callbacks.

Q23. What is a ternary operator?

condition ? valueIfTrue : valueIfFalse. Concise alternative to if/else for expressions. Don't nest deeply — hurts readability.

Q24. What is optional chaining?

?. safely accesses nested properties. Returns undefined instead of throwing if any part is null/undefined. obj?.prop, obj?.method?(), arr?.[0].

Q25. What is nullish coalescing?

?? returns the right side only if the left side is null or undefined. Unlike ||, it doesn't trigger on 0 or "".

Q26. What is the difference between for...in and for...of?

`for...in` iterates over enumerable property KEYS (strings). `for...of` iterates over VALUES of iterables (arrays, strings, etc.). Don't use `for...in` on arrays.

Q27. What is hoisting?

JavaScript engine moves variable and function declarations to the top of their scope during the creation phase. `var` is hoisted as `undefined`; `let/const` are hoisted in TDZ; function declarations are fully hoisted.

Q28. What is the Temporal Dead Zone?

The period between the start of a block scope and a `let/const` declaration. Accessing the variable during TDZ throws `ReferenceError`.

Q29. What is scope?

The region of code where a variable is accessible. Types: global scope, function scope (`var`), block scope (`let/const`). JavaScript uses lexical scoping.

Q30. What is a closure?

A function that has access to its outer scope's variables even after the outer function has returned. Created naturally when inner functions reference outer variables.

Q31. What is the difference between a function declaration and expression?

Declaration: `function foo(){} — hoisted fully`. Expression: `const foo = function(){} — only variable hoisted, not function`.

Q32. What is an IIFE?

Immediately Invoked Function Expression: `(function(){}())`. Creates an isolated scope, runs immediately. Used to avoid polluting global scope.

Q33. What does 'use strict' do?

Enables strict mode. Disallows: undeclared variables, duplicate parameters, writing to read-only properties, this being global object in functions. Errors are thrown instead of silently failing.

Q34. What is event delegation?

Attaching ONE event listener to a parent element to handle events from multiple children, using `event.target` to identify the source. More efficient than per-element listeners.

Q35. What is the DOM?

Document Object Model — a tree representation of the HTML page that JavaScript can manipulate. `document.getElementById()`, `querySelector()`, etc. provide access.

Q36. What is JSON?

JavaScript Object Notation — a text format for structured data. `JSON.stringify(obj)` converts JS to JSON string. `JSON.parse(str)` converts JSON string back to JS object. Keys must be double-quoted strings.

Q37. What is localStorage?

Browser API for persisting key-value string data. Persists across browser sessions. `localStorage.setItem(key, val)`, `getItem(key)`, `removeItem(key)`. Max ~5MB. Synchronous. No server access.

Q38. What is the difference between synchronous and asynchronous code?

Synchronous: code runs line by line, each operation must complete before the next. Asynchronous: operations can be initiated and the program continues while waiting. `setTimeout`, `fetch`, `promises` are async.

Q39. What is a Promise?

An object representing eventual completion/failure of an async operation. States: `pending`, `fulfilled`, `rejected`. Use `.then()` for success, `.catch()` for errors, `.finally()` for cleanup.

Q40. What is async/await?

Syntactic sugar over Promises. `async` makes a function return a Promise. `await` pauses execution until a Promise resolves. Makes `async` code look synchronous. Use `try/catch` for error handling.

Q41. How do you copy an object?

Shallow: `{...obj}` or `Object.assign({}, obj)`. Deep: `structuredClone(obj)`, `JSON.parse(JSON.stringify(obj))` for JSON-compatible data, or `lodash_.cloneDeep()`.

Q42. What is the event loop?

Mechanism that allows JS to be non-blocking. Call stack runs sync code. Web APIs handle async tasks. Microtask queue (Promises) drains before macrotask queue (`setTimeout`). Event loop picks tasks when call stack is empty.

Q43. What does Number() do to different types?

`Number(null)=0`, `Number(undefined)=NaN`, `Number("")=0`, `Number(' ')=0`, `Number(true)=1`, `Number(false)=0`, `Number('42')=42`, `Number('abc')=NaN`, `Number([])=0`, `Number([1])=1`.

Q44. What is string immutability?

Strings cannot be changed after creation. `str[0]='X'` silently fails. All string methods return new strings. The variable can be reassigned to a new string, but the original string value doesn't change.

Q45. What does indexOf return if not found?

-1. This is why many prefer `includes()` for existence checks — it returns `true/false` which is more readable.

Q46. How do you convert a string to an array?

`str.split("")` or `[...str]` or `Array.from(str)`. `split("")` splits on every character. For Unicode: `Array.from` or spread handle multi-byte chars correctly.

Q47. What is the difference between null and undefined in JSON?

`JSON.stringify` converts `undefined` to nothing (key is omitted). `null` becomes the string `'null'`. `undefined` values in objects are removed; in arrays they become `null`.

Q48. What is Object.keys()?

Returns an array of an object's own enumerable property names (keys) as strings. Doesn't include prototype properties or Symbol keys.

Q49. What is the rest parameter?

`...params` in function definition collects remaining arguments into an array. Must be last parameter. Unlike arguments, it's a real array.

Q50. What is a default parameter?

function `fn(a, b=10)` — `b` defaults to 10 if not provided or `undefined`. Default is evaluated at call time. Can reference earlier parameters.

Q51. How do you remove duplicates from an array?

`[...new Set(arr)]` — Set only stores unique values. Convert back to array with spread. For objects, need custom logic as objects compare by reference.

Q52. What is the difference between == false and !value?

`== false` uses type coercion: `" == false (true)`, `0 == false (true)`, `[] == false (true)`. `!value` checks falsiness: `!" (true)`, `!0 (true)`, `![] (false — [] is truthy!)`. `[] == false` is true but `![]` is false.

Q53. What does Array.from do?

Creates a new array from array-like (length + indices) or iterable objects. `Array.from('abc')=['a','b','c']`. `Array.from({length:3}, (_,i)=>i)=[0,1,2]`. Useful for `NodeList`, `Set`, `Map`, arguments.

Q54. What is a Set?

Collection of unique values. `new Set([1,1,2,3]) = Set{1,2,3}`. `.add()`, `.has()`, `.delete()`, `.size`. Iteration order is insertion order. Great for deduplication.

Q55. What is a Map?

Key-value collection where keys can be any type. Better than plain object for non-string keys. `.set(key,val)`, `.get(key)`, `.has(key)`, `.delete(key)`, `.size`. Preserves insertion order.

Q56. What is the difference between Map and Object?

`Map`: any type as key, size property, direct iteration, better performance for frequent add/delete. `Object`: string/Symbol keys, must use `Object.keys()`, JSON serialisable, prototype chain.

Q57. What is short-circuit evaluation?

`&&`: stops at first falsy value and returns it. `||`: stops at first truthy value and returns it. Used for conditionals without `if`: `condition && doSomething()`.

Q58. What does Array.isArray() do?

Returns true if value is an Array, false otherwise. More reliable than `typeof` (which returns 'object') or `instanceof` (which fails across iframes).

Q59. What is the difference between slice and substring for strings?

`slice`: supports negative indices (counts from end). `substring`: treats negatives as 0, swaps start/end if start > end. Prefer `slice`.

Q60. How do you check if an object has a property?

'key' in obj (checks own + inherited). `obj.hasOwnProperty('key')` or `Object.hasOwn(obj,'key')` (own only). `obj.key !== undefined` (works but can't distinguish from explicitly set undefined).

Q61. What is string interpolation?

Embedding expressions inside strings using template literals: ``Hello ${name}, you are ${age} years old``. More readable than `'Hello ' + name + ', you are ' + age`.

Q62. What is an object method shorthand?

`const obj = { greet() { return 'hi'; } }` instead of `{ greet: function() { return 'hi'; } }`. Arrow shorthand: `greet: () => 'hi'` (note: different this behavior).

Q63. What does sort() do without arguments?

Sorts array IN PLACE, converting elements to strings and sorting lexicographically. `[10,1,21].sort()` gives `[1,10,21]` (because `'1' < '10' < '21'` as strings). Always provide compare function for numbers.

Q64. How do you find the maximum value in an array?

`Math.max(...arr)` or `Math.max.apply(null, arr)`. For arrays with objects: `arr.reduce((max, x) => x.val > max.val ? x : max)`. Not `arr.sort()` — $O(n \log n)$ is worse than $O(n)$.

Q65. What is the typeof NaN?

'number'. NaN is technically of type number in JavaScript — it represents a numeric result that is undefined or not representable.

Q66. What are truthy values?

Everything that isn't falsy. Includes: non-zero numbers, non-empty strings, objects, arrays (even empty), functions. Tricky: `'0'`, `' '`, `[]`, `{}`, `-1` are ALL truthy.

Q67. What does `Number.isInteger()` do?

Returns true if value is a finite integer (no decimal). `Number.isInteger(4.0)` is true (`4.0 === 4` in JS). `Number.isInteger(4.5)` is false.

Q68. What is the difference between `concat` and `spread` for arrays?

Both create new arrays. `arr1.concat(arr2)` is the method form. `[...arr1, ...arr2]` is the spread form. Spread is more flexible and generally preferred in modern code.

Q69. What does `flat()` do?

Returns a new array with nested arrays flattened to specified depth. `[1,[2,[3]]].flat()` = `[1,2,[3]]`. `[1,[2,[3]]].flat(Infinity)` = `[1,2,3]`. Does not mutate original.

Q70. What is the difference between `parseInt` and `Number` for strings?

`Number('42px')` = `NaN` (strict). `parseInt('42px')` = 42 (stops at first non-digit). `parseInt('0x1f', 16)` = 31 (hex). Always pass radix to `parseInt`: `parseInt(str, 10)`.

Q71. What is a named function expression?

`const factorial = function fact(n) { return n <= 1 ? 1 : n * fact(n-1); }`. The name 'fact' is only accessible inside the function body — useful for recursion and stack trace readability.

Q72. What is the difference between `pass-by-value` and `pass-by-reference`?

Primitives are passed by value — function receives a copy, changes don't affect original. Objects/arrays are passed by reference — function gets reference to same object, mutations affect original. The reference itself is passed by value (can't reassign caller's variable).

Q73. What is function hoisting?

Function declarations are hoisted to the top of their scope with their full body — they can be called before the declaration in code. Function expressions are NOT fully hoisted — only the variable declaration is.

Q74. What does the `delete` operator do?

Removes an own property from an object. `delete obj.prop`. Returns true (even if property doesn't exist). Cannot delete variables, function declarations, or non-configurable properties.

Q75. How do you iterate over an object's entries?

`Object.entries(obj).forEach(([key, val]) => ...)` or for `(const [key,val] of Object.entries(obj))`. `Object.keys` for just keys, `Object.values` for just values.

Q76. What is a pure function?

A function that: 1) always returns the same output for same input, 2) has no side effects (doesn't modify external state, no I/O). Pure functions are predictable, testable, and safe to optimise.

Q77. What is a side effect in programming?

Any modification of state outside the function's scope: modifying a variable, writing to a file, making a network request, logging to console. Functions with side effects are harder to test and reason about.

Q78. How do you convert an array to a string?

`arr.join(separator)` — default separator is comma. `[1,2,3].join('-')` = `'1-2-3'`. `arr.toString()` uses comma. `JSON.stringify(arr)` for JSON format.

Q79. What is `Array.prototype.fill()`?

Fills all (or range of) array elements with a static value. `new Array(5).fill(0)` = `[0,0,0,0,0]`. `arr.fill('x', 2, 4)` fills indices 2-3 with 'x'. Mutates original.

Q80. What is the purpose of `.toString(radix)` on numbers?

Converts number to string in specified base. (255).toString(16) = 'ff' (hex). (8).toString(2) = '1000' (binary). (255).toString(10) = '255' (decimal).

Q81. What is a WeakRef?

Holds a weak reference to an object — doesn't prevent garbage collection. weakRef.deref() returns the object or undefined if collected. Used for caches that should auto-clear.

Q82. What is globalThis?

A universal way to access the global object regardless of environment. In browser: window. In Node.js: global. globalThis works in both, introduced in ES2020.

Q83. How do you check if a number is finite?

Number.isFinite(value) — returns true only for finite numbers. isFinite() (global) coerces first (isFinite('5') is true). Number.isFinite(Infinity) = false, Number.isFinite(NaN) = false.

Q84. What is string padding?

'5'.padStart(3, '0') = '005'. 'hi'.padEnd(5, '.') = 'hi...'. Used for formatting: zero-padding numbers, aligning text.

Q85. What does Array.prototype.includes() do?

Returns true/false for value existence. Uses SameValueZero algorithm — can find NaN (unlike indexOf). [1,NaN].includes(NaN) = true. [1,NaN].indexOf(NaN) = -1.

Q86. What is computed property syntax?

const key='name'; const obj={{[key]:'Alice'}} — property name computed at runtime. Useful for dynamic keys: const ops = {[operator](a,b){...}}.

Q87. What does Object.entries() return?

An array of [key, value] pairs from the object's own enumerable string-keyed properties. Useful for iteration and converting to Map: new Map(Object.entries(obj)).

Q88. What is a tagged template literal?

fn`template \${expr}` — calls fn with (strings, ...values) where strings is array of static parts and values are interpolated expressions. Used in styled-components, GraphQL, SQL.

Q89. What does Number.isNaN() differ from isNaN()?

isNaN() coerces the argument: isNaN('hello') = true (converts to NaN first). Number.isNaN() strictly checks: Number.isNaN('hello') = false. Only true for the actual NaN value.

Q90. How do you flatten an array of arrays?

[].concat(...arr) or arr.flat() or arr.reduce((acc, a) => acc.concat(a), []) for one level. arr.flat(Infinity) for all levels.

Q91. What is Object.freeze() vs Object.seal()?

freeze: no add, delete, or modify. seal: no add or delete but can modify existing. Neither is deep (nested objects remain mutable).

Q92. What is the arguments object?

Array-like object available inside regular functions containing all passed arguments. Not available in arrow functions. Not a real array — use Array.from(arguments) or rest params (...args) instead.

Q93. What is a constructor function?

A regular function intended to be called with new to create objects. By convention, name starts with capital. new sets this to new object and returns it. Replaced by class syntax in ES6.

Q94. What is String.prototype.at()?

ES2022 method that supports negative indices. 'hello'.at(-1) = 'o'. Equivalent to str[str.length-1] but cleaner.

Q95. What does flatMap() do?

Combines map() and flat(1) in one pass. More efficient than mapping then flattening. `[1,2,3].flatMap(n=>[n,n*2]) = [1,2,2,4,3,6]`.

Q96. What is the difference between for...of and forEach?

for...of works on any iterable, supports break/continue/return/await. forEach only works on arrays, no break/continue support, ignores return values.

Q97. What are Symbol.iterator and the iteration protocol?

An object is iterable if it has a [Symbol.iterator] method returning an iterator (object with next() method returning {value, done}). for...of uses this protocol. Arrays, strings, Maps, Sets are iterable.

Q98. What is Object.fromEntries()?

Transforms [key, value] pairs into an object. Inverse of Object.entries(). Object.fromEntries(map) converts Map to object. Object.fromEntries(entries.filter(predicate)) filters object properties.

Q99. What does Array.prototype.some() return?

true if at least one element passes the callback test, false otherwise. Short-circuits on first truthy result. `[1,2,3].some(n=>n>2) = true` (short-circuits at 3).

Q100. What is the optional catch binding?

ES2019: try { ... } catch { ... } — you can omit the error parameter if you don't need it. Previously required: catch(e) even if unused.

Q101. How do you check if an object is empty?

Object.keys(obj).length === 0. Or Object.entries(obj).length === 0. Not JSON.stringify(obj) === '{}' — unreliable for objects with non-serialisable values.

Section B: 75 Intermediate Questions

Q1. Explain the difference between shallow copy and deep copy.

Shallow copy: copies only the top level — nested objects are still shared references. Methods: {...obj}, Object.assign({}, obj), arr.slice(). Deep copy: recursively copies all levels — no shared references. Methods: structuredClone(obj) [modern], JSON.parse(JSON.stringify(obj)) [JSON-compatible], lodash _cloneDeep().

Q2. What is the output of 0.1 + 0.2 and why?

0.30000000000000004. JavaScript uses IEEE 754 double-precision floating point. 0.1 and 0.2 cannot be represented exactly in binary — they have infinite repeating fractions. The accumulated error causes the result to slightly exceed 0.3. Fix: toFixed(1), Math.round(result * 10) / 10, or use integers (multiply by 100, work in cents).

Q3. What is the event loop and how does it work?

JS is single-threaded. Event loop monitors call stack and task queues. When call stack is empty: 1) Drain microtask queue completely (Promises, queueMicrotask). 2) Take ONE macrotask (setTimeout, I/O). 3) Render frame (browser). 4) Repeat. This is why Promises (.then) always fire before setTimeout callbacks.

Q4. What will this print? console.log(typeof typeof 42)

'string'. typeof 42 evaluates to the string 'number'. typeof 'number' (a string) evaluates to 'string'. typeof always returns a string, so typeof of any typeof expression is always 'string'.

Q5. What is the prototype chain and how does property lookup work?

Every JS object has [[Prototype]] pointing to another object. Property access walks this chain: own properties first, then [[Prototype]], then [[Prototype]]'s [[Prototype]], until null. If not found: undefined. Functions share methods via

prototype chain — that's how `dog.toString()` works without `toString` defined on `dog`.

Q6. How does 'new' work internally?

1) Creates a new empty object. 2) Sets the object's `[[Prototype]]` to `Constructor.prototype`. 3) Calls the Constructor function with `this = new object`. 4) Returns the new object — unless Constructor returns a different object, in which case that is returned.

Q7. What is the difference between call stack and event loop?

Call stack: executes code synchronously, LIFO. Tracks execution context. One item at a time. Event loop: mechanism that moves callbacks from queues to the call stack when it's empty. They work together: call stack runs, event loop feeds it new work when idle.

Q8. What is Promise chaining and what does each `.then` return?

Each `.then()` returns a new Promise. If the callback returns a value, the new Promise resolves with that value. If it returns a Promise, the chain waits for that Promise to settle. If it throws, the rejection propagates to the next `.catch()`. This creates a flat pipeline for sequential async operations.

Q9. Explain closures with a practical example.

A closure is a function that retains access to its outer scope after the outer function has returned. Practical use:

```
function createCounter(start=0) { let count=start; return { inc(){count++;}, get(){return count;} }; }
```

 — `count` is private, only accessible via the returned methods.

Q10. What is the output: `const arr = [1,2,3]; arr[10] = 11; console.log(arr.length)?`

11. Setting `arr[10]` creates a sparse array. Indices 3-9 are 'empty holes' (not undefined). `length` tracks the highest index + 1. Iterating with `forEach/map` skips empty slots; `for...of` and `[...arr]` give undefined for holes.

Q11. What is currying?

Transforming a function of multiple arguments into a sequence of functions each taking one argument. `add(1)(2)(3)` instead of `add(1,2,3)`. Enables partial application and functional composition. `const curry = fn => a => b => fn(a,b)`.

Q12. What is memoisation?

Caching function results for previously seen inputs. The function returns the cached result instead of recomputing.

```
function memoize(fn){ const cache=new Map(); return (...args)=>{ const k=JSON.stringify(args); if(!cache.has(k)) cache.set(k,fn(...args)); return cache.get(k); }; }
```

Q13. What is the difference between debounce and throttle?

Debounce: executes function after N ms of silence since last call. Good for search autocomplete. Throttle: executes at most once per N ms regardless of calls. Good for scroll/resize. Debounce waits for calm; throttle guarantees max frequency.

Q14. What are generators?

Functions with `*` that can pause (`yield`) and resume.

```
function* gen(){ yield 1; yield 2; }
```

 The generator object has `.next()` which returns `{value, done}`. Used for lazy sequences, custom iterators, and was the foundation of `async/await` (now replaced).

Q15. What is the difference between Map and WeakMap?

Map: any type as key, iterable, has `.size`, keys are strongly held (prevent GC). WeakMap: object keys only, not iterable, no `.size`, keys are weakly held (allow GC). Use WeakMap for associating private data with objects without preventing cleanup.

Q16. How does `Array.prototype.reduce` work internally?

`reduce(callback, initialValue)` starts accumulator at `initialValue` (or first element if no initial value — changes start index to 1). For each element: `accumulator = callback(accumulator, currentValue, index, array)`. Returns final

accumulator. Always provide initial value to avoid bugs on empty arrays.

Q17. What is `Object.defineProperty`?

Defines or modifies a property with full control over its descriptor: value, writable, enumerable, configurable. `Object.defineProperty(obj, 'x', {get(){...}, set(v){...}})`. Used by Vue.js reactivity system (Vue 2), property validation, read-only properties.

Q18. Explain the Proxy object with an example.

Proxy wraps an object and intercepts operations. `const proxy = new Proxy(target, {get(t,k){console.log(k); return t[k]})`. Useful for: validation, logging, virtual properties, defensive programming. Vue 3 uses Proxy for its reactivity system.

Q19. What is async iteration?

for await...of loops over async iterables (objects with `[Symbol.asyncIterator]`). Each iteration awaits the next value. Used for streaming data: `for await (const chunk of readableStream) {}`. Requires async context.

Q20. What happens if you await a non-Promise?

The value is wrapped in `Promise.resolve()`. `await 42` gives 42; `await null` gives null. This is why `await` works consistently regardless of whether a function is sync or async.

Q21. What is `Promise.allSettled` vs `Promise.all`?

`Promise.all`: rejects immediately on first rejection. `Promise.allSettled`: waits for ALL promises, returns `[[status:'fulfilled', value] | {status:'rejected', reason}]` for each. Use `all` when all must succeed; `allSettled` when partial failure is OK.

Q22. What is the `Symbol.toPrimitive` used for?

Customises type conversion of objects. `const obj = { [Symbol.toPrimitive](hint){ if(hint==='number') return 42; return 'obj'; } }`. `+obj` gives 42; ``$${obj}`` gives 'obj'. Called before `valueOf/toString`.

Q23. What is a WeakSet?

Collection of weakly held object references. Only stores objects (not primitives). Not iterable, no `.size`. Useful for tracking objects without preventing GC — e.g., marking DOM nodes as processed: `const visited = new WeakSet(); visited.add(node)`.

Q24. What is tail call optimisation?

TCO: when a function's last operation is a recursive call, the engine can reuse the current stack frame instead of creating a new one, preventing stack overflow. ES6 specified TCO but most engines only implement in strict mode. Trampolining is a practical alternative.

Q25. What is the structured clone algorithm?

`structuredClone(obj)` performs deep cloning. Handles circular references, Date, Map, Set, ArrayBuffer, TypedArrays. Does NOT handle functions, DOM nodes, class instances (loses prototype). Better than `JSON.parse/stringify` for most deep copy needs.

Q26. Explain how `async/await` is compiled under the hood.

`async/await` is syntactic sugar over Promises and generators. An `async` function returns a Promise. Each `await` is a yield point in an internal generator. The runtime wraps the generator in a Promise executor, calling `.next()` when each awaited Promise resolves. This entire mechanism uses `Promise.then()` chains.

Q27. What is the difference between a microtask and a macrotask?

Microtask: higher priority, runs after current operation and BEFORE the next render/macrotask. Sources: `Promise.then`, `queueMicrotask`, `MutationObserver`. Macrotask: scheduled at next available loop iteration. Sources: `setTimeout`, `setInterval`, I/O. ALL microtasks drain before any macrotask runs.

Q28. How would you implement Promise.all from scratch?

```
function myAll(promises){ return new Promise((res, rej) => { const results = []; let remaining = promises.length; if (!remaining) return res([]); promises.forEach((p, i) => Promise.resolve(p).then(v => { results[i]=v; if(--remaining===0) res(results); }).catch(rej)); }); }
```

Q29. What is the iterator protocol?

An object is an iterator if it has a next() method returning {value, done}. An object is iterable if it has [Symbol.iterator]() returning an iterator. Arrays, strings, Maps, Sets implement this. for...of, spread, destructuring all consume iterables via this protocol.

Q30. What is the AbortController and AbortSignal?

AbortController cancels async operations. const ctrl = new AbortController(); fetch(url, {signal: ctrl.signal}); ctrl.abort() cancels the fetch. The signal can be passed to multiple operations. Useful for cancelling obsolete requests in search autocomplete.

Q31. What is the Intl object?

JavaScript's internationalisation API. Intl.NumberFormat('de-DE').format(1234567) = '1.234.567'. Intl.DateTimeFormat('ja-JP').format(new Date()). Intl.Collator for locale-aware string sorting. Intl.RelativeTimeFormat for '3 days ago' text.

Q32. Explain the difference between enumerable, configurable, and writable property descriptors.

enumerable: shows in for...in, Object.keys, JSON.stringify. configurable: can descriptor be changed; can property be deleted. writable: can value be changed. Object.defineProperty({}, 'x', {value:1, writable:false, enumerable:false, configurable:false}) creates non-enumerable, non-writable, non-deletable property.

Q33. What is prototype pollution and how do you prevent it?

An attack where an attacker adds properties to Object.prototype via user input: obj['__proto__']['isAdmin'] = true. Affects ALL objects. Prevention: validate user input keys (reject __proto__, constructor, prototype), use Object.create(null) for data stores, use Map instead of plain objects for user data.

Q34. What is the difference between Object.keys, Object.getOwnPropertyNames, and Reflect.ownKeys?

Object.keys: enumerable own string keys. Object.getOwnPropertyNames: all own string keys (including non-enumerable). Reflect.ownKeys: all own keys including Symbols. Use getOwnPropertyNames for full reflection, Reflect.ownKeys when you need Symbols too.

Q35. How does JavaScript handle integer overflow?

Regular numbers (64-bit float) lose precision above Number.MAX_SAFE_INTEGER (2⁵³ - 1 = 9007199254740991). 9007199254740992 === 9007199254740993 is true! Use BigInt for precise large integer arithmetic: 9007199254740992n + 1n = 9007199254740993n.

Q36. What is a Proxy trap and what traps are available?

A trap is a handler method that intercepts an operation on the target object. Available traps: get, set, has (in operator), deleteProperty, apply (function calls), construct (new), getPrototypeOf, setPrototypeOf, isExtensible, preventExtensions, getOwnPropertyDescriptor, defineProperty, ownKeys.

CHAPTER
13

Cheat Sheet & Quick Revision

All the key concepts, syntax, and patterns in one place — your last-minute interview reference.

Variables Quick Reference

Variables

```
var   → function scope · hoisted (undefined) · can redeclare · attaches to window
let   → block scope   · hoisted (TDZ)         · no redeclare · use for mutable values
const → block scope   · hoisted (TDZ)         · no redeclare · no reassign (object contents CAN change)
Rule: const > let > var (default to const)
```

Data Types Quick Reference

Type	Example	typeof	Notes
Number	42, 3.14, NaN	'number'	IEEE 754 float; NaN is type number
String	'hi', `\${x}`	'string'	Immutable; Unicode UTF-16
Boolean	true, false	'boolean'	Falsy: false, 0, "", null, undef, NaN
Null	null	'object'	BUG — check with === null
Undefined	undefined	'undefined'	Unassigned variables
Symbol	Symbol('id')	'symbol'	Unique; not enumerable
BigInt	42n, BigInt(42)	'bigint'	No precision loss for large ints
Object	{}, [], fn(){}	'object'	By reference; Array.isArray()

Operators Cheat Sheet

```
// EQUALITY
```

```
=== // strict (always use)  == // loose (avoid)
```

```
!== // strict not equal    != // loose not equal
```

```
// LOGICAL (return actual values, not just true/false)
```

```
&& // first falsy OR last value    || // first truthy OR last value
```

```
?? // first non-null/undefined    ! // negation
```

```
// MODERN (ES2020+)
```

```
?. // optional chaining – safe access through null/undefined
```

```
??= // assign if null/undefined    ||= // assign if falsy
```

```
// TYPE COERCION TRAPS

0 == false → true | '' == false → true | [] == false → true

null == undefined → true | NaN == NaN → false

null >= 0 → true BUT null > 0 → false AND null == 0 → false
```

Array Methods Reference

Method	Mutates?	Returns	Key Notes
push(...items)	YES	New length	Add to end
pop()	YES	Removed item	Remove from end
unshift(...items)	YES	New length	Add to start
shift()	YES	Removed item	Remove from start
splice(s,d,...i)	YES	Removed items[]	Insert/remove anywhere
sort(fn)	YES	Same array	Always use compare fn for numbers
reverse()	YES	Same array	Reverses in place
fill(v,s,e)	YES	Same array	Fill range with value
map(fn)	NO	New array	Transform each element
filter(fn)	NO	New array	Keep matching elements
reduce(fn,init)	NO	Any type	Accumulate to single value
slice(s,e)	NO	New array	Copy sub-array
flat(depth)	NO	New array	Flatten nested arrays
flatMap(fn)	NO	New array	Map + flat(1)
find(fn)	NO	First match/undef	First matching element
findIndex(fn)	NO	Index or -1	Index of first match
some(fn)	NO	boolean	Any element matches?
every(fn)	NO	boolean	All elements match?
includes(v)	NO	boolean	Value exists? (handles NaN)
indexOf(v)	NO	Index or -1	First index (no NaN support)
forEach(fn)	NO	undefined	Side effects; can't break
concat(...arrs)	NO	New array	Merge arrays
join(sep)	NO	String	Array to string

String Methods Reference

Method	Returns	Notes
toUpperCase/Lower()	new string	Case conversion
trim/Start/End()	new string	Remove whitespace
split(sep)	array	String to array
slice(s,e)	new string	Supports negative indices
indexOf(str)	number	First position or -1
includes(str)	boolean	Cleaner than indexOf
startsWith/endsWith()	boolean	Prefix/suffix check
replace(s,r)	new string	First match; use regex /g for all
replaceAll(s,r)	new string	All matches (ES2021)
padStart/End(n,fill)	new string	Zero-pad numbers
repeat(n)	new string	Repeat string n times
match(regex)	array/null	Regex match
charAt(i) / at(i)	char	at() supports negative
charCodeAt(i)	number	Unicode code point
String.fromCharCode()	string	Code to character

Scope & Hoisting Quick Reference

Scope & Hoisting

SCOPE:
Global → accessible everywhere
Function → var lives here
Block → let/const live here (any {})
HOISTING:
function declaration → fully hoisted (call before definition: OK)
var → hoisted as undefined (use before: undefined)
let / const → hoisted in TDZ (use before: ReferenceError)
function expression → only variable hoisted (let=TDZ, var=undefined)

Common Interview Traps

Code	Output	Why
typeof null	'object'	Historical bug

<code>0.1 + 0.2 === 0.3</code>	false	IEEE 754 floating point
<code>[] == false</code>	true	<code>[] → " → 0 === 0</code>
<code>![]</code>	false	<code>[]</code> is truthy
<code>null >= 0</code>	true	<code>null → 0</code> in comparison
<code>null == 0</code>	false	<code>null</code> only equals <code>null/undefined</code>
<code>NaN === NaN</code>	false	<code>NaN</code> is unique
<code>'1' + 2 + 3</code>	<code>'123'</code>	<code>+</code> coerces to string
<code>1 + 2 + '3'</code>	<code>'33'</code>	Left-to-right: <code>1+2=3</code> , <code>3+'3'='33'</code>
<code>typeof typeof 42</code>	<code>'string'</code>	<code>typeof</code> always returns string
<code>[1,2].sort()</code>	<code>[1,2]</code>	OK here but <code>[10,2].sort()=[10,2]!</code>
<code>new Array(3)</code>	<code>sparse[3]</code>	3 empty slots, NOT <code>[3]</code>
<code>[...Array(3).keys()]</code>	<code>[0,1,2]</code>	Generate range

Truthy & Falsy Reference

```
// FALSY (7 values – memorise these!)
false, 0, -0, 0n, '', null, undefined, NaN

// TRUTHY TRAPS – these are ALL truthy:
'0' ' ' ' ' [] {} -1 Infinity function(){} new Date()

// Boolean conversion:
!!value // double negation to boolean
Boolean(val) // explicit conversion
```

Modern JavaScript (ES6+) Quick Reference

Feature	Syntax
Destructuring	<code>const {a,b} = obj; const [x,y] = arr;</code>
Spread	<code>const copy = {...obj}; const arr2 = [...arr];</code>
Rest	<code>function fn(...args) {} const {a, ...rest} = obj;</code>
Arrow functions	<code>const fn = (x) => x * 2;</code>
Template literals	<code>`Hello \${name}!`</code> — multi-line, expressions
Default params	<code>function fn(x, y = 10) {}</code>
Optional chaining	<code>obj?.prop?.nested arr?.[0] fn?.()</code>

Nullish coalescing	<code>x ?? 'default'</code> — only null/undefined triggers
Logical assignment	<code>x ??= 5; x = 5; x &&= 5;</code>
Class syntax	<code>class Animal { #private = 0; get count(){} }</code>
Modules	<code>import {fn} from './mod.js'; export const x = 1;</code>
Promise.all/any	<code>await Promise.all([p1, p2]);</code>
async/await	<code>async function fn() { const x = await fetch(url); }</code>
structuredClone	<code>structuredClone(obj)</code> — deep clone
BigInt	<code>const big = 9007199254740993n;</code>
Symbol	<code>const id = Symbol('unique key');</code>

Closing Notes

You have completed JavaScript Complete Handbook — Part 1.

The fundamentals covered here are the foundation of everything in JavaScript. Closures, scope, hoisting, type coercion, and the event loop — these concepts appear in every interview and every production codebase. Master the fundamentals deeply before moving to frameworks.

Continue with **Part 2: Advanced JavaScript & Internals** for execution contexts, prototypes, async patterns, and 225+ more interview questions.

Practice daily. Build projects. Read error messages carefully.